

Programmation

Le programme est découpé en trois fonctions principales pour le fonctionnement normal du programme, en plus de deux autres fonctions qui aident à l'évaluation de la performance.

La fonction `main` s'assure que les arguments nécessaires au fonctionnement du programme sont présents et essaye de les lire puis assigne leurs valeurs à des variables qui seront utilisées par les autres fonctions. Elle appelle la fonction `countWordsConcurrently(text, segmentSize)` et imprime le total de mots qu'elle retourne.

La fonction `countWordsConcurrently()` prend le texte passé en argument et le divise en segment de la taille passée en argument, puis assure la synchronisation des calculs à l'aide d'un `WaitGroup`. Spécifiquement, on crée un channel où le total de chaque segment sera accumulé et un `waitgroup` qui assure que toutes les routines seront synchronisées. Puis, on parcourt le texte à tous les `segmentSize` caractères, et pour chaque segment on crée une nouvelle routine à laquelle on passe en argument l'indice de départ et de fin du segment. Cette routine commence par `defer l'instruction Done()` afin de s'assurer qu'aucune routine n'est laissée en attente indéfiniment si une erreur se produit, puis elle appelle `countSegmentWords`. Une fois tous les segments du texte traités, on appelle une dernière goroutine qui attend la fin de toutes les autres goroutine puis qui ferme le channel de résultats. Finalement, on additionne tous les résultats ayant été passé dans le channel et on retourne ce total.

Finalement, la fonction `countSegmentWords()` prend un tableau de caractères et les indices de début et de fin du segment à compter, et elle compte ensuite le nombre de mots à l'intérieur des limites des indices. On vérifie si le segment se termine au milieu d'un mot, et si c'est le cas on ne compte pas ce dernier mot, il sera plutôt compté comme le premier mot du segment suivant. On s'assure de ne pas avoir calculé de valeur négative, ce qui ne devrait pas être possible de toute façon, puis on retourne le nombre de mots calculé comme résultat.

En ce qui concerne les fonctions servant aux tests et à l'analyse de performance, la fonction la plus utilisée est `generateTestText(size)` qui permet de générer puis retourner un texte de `size` caractères contenant une répétition des mots "hello", "world", "test", "concurrent", "goroutine", "channel", "segment", "performance", "benchmark" et "analysis" jusqu'à ce que la limite de caractères soit atteinte, et sans la dépasser.

De plus, afin d'obtenir plus d'information sur la performance, on a demandé à l'IA générative de générer une fonction pour pouvoir observer comment la performance est affectée par la variation de taille des segments sur des fichiers de taille différente. Cette fonction utilise une structure afin d'organiser les tests selon la taille des textes puis la taille des segments. Pour chaque test, on génère un texte avec la fonction `generateTestText()` et on calcule le nombre de segments total. On lance ensuite la fonction `countWordsConcurrently()` une fois pour la mettre en mémoire récente pour que les tests qui suivent soient faits sur un meilleur pied d'égalité. On calcule ensuite le délai de traitement pour faire rouler la fonction `countWordsConcurrently()` dix fois sur le texte de cette taille avec des segments de cette taille. Avec ce délai on calcule le temps moyen de traitement de la fonction avec ces paramètres et la vitesse de traitement en mot/ms. Le tout est ensuite imprimé sur la console. Pour utiliser cette fonction, il suffit de remplacer le contenu de la fonction `main` par un appel de `analyzePerformance()`.

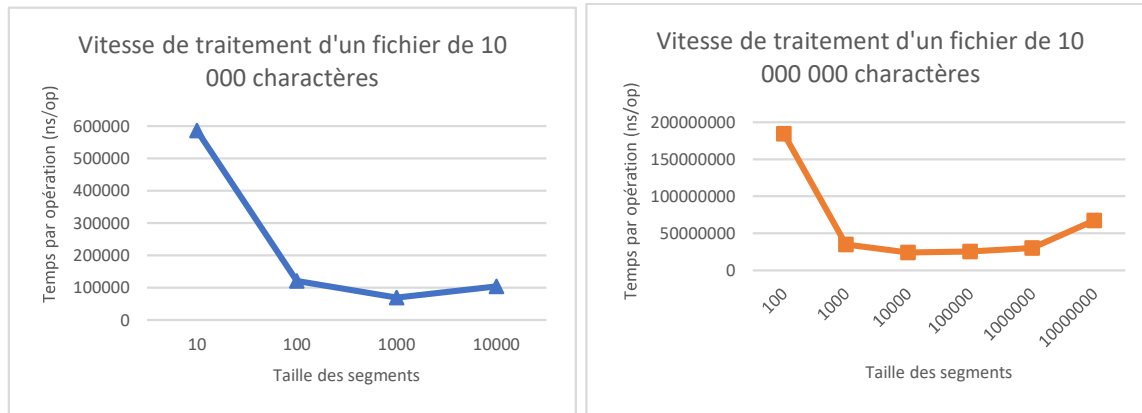
En ce qui concerne les tests, on teste huit cas : un fichier vide, un fichier contenant un seul mot, un fichier contenant plusieurs lignes de plusieurs mots, des segments d'une taille qui coupe au milieu d'un mot, des segments qui ne sont que d'un caractère, des segments qui sont aussi longs que le texte, un fichier qui contient des espaces en trop avant et après le texte, et un fichier qui contient des espaces en trop entre les mots du texte. Aucun cas ne devrait générer une erreur. Ces cas sont testés directement en appelant la fonction `countWordsConcurrently`, ce qui teste automatiquement la fonction `countSegmentWords` qui ne doit être appelée que dans le contexte de `countWordsConcurrently`.

Pour le benchmark, on évalue deux tailles de texte, un de taille moyenne (10ko) et un très grand (10Mo). On évalue le grand texte sur un plus grand éventail de valeur afin de pouvoir comparer ses résultats selon le nombre de segments et la taille des segments avec ceux du texte de 10ko. De plus cela nous permet aussi de mieux comparer ces résultats à ceux obtenus avec `analyzePerformance()`. La taille de chaque texte est encodée dans une constante et on crée une fonction unique pour chaque couple (texte, taille de segment) qu'on évalue. On utilise encore directement `countWordsConcurrently`, puisque la performance à mesurer est la vitesse de traitement simultané, et pas la capacité à compter des mots du programme.

Résultats

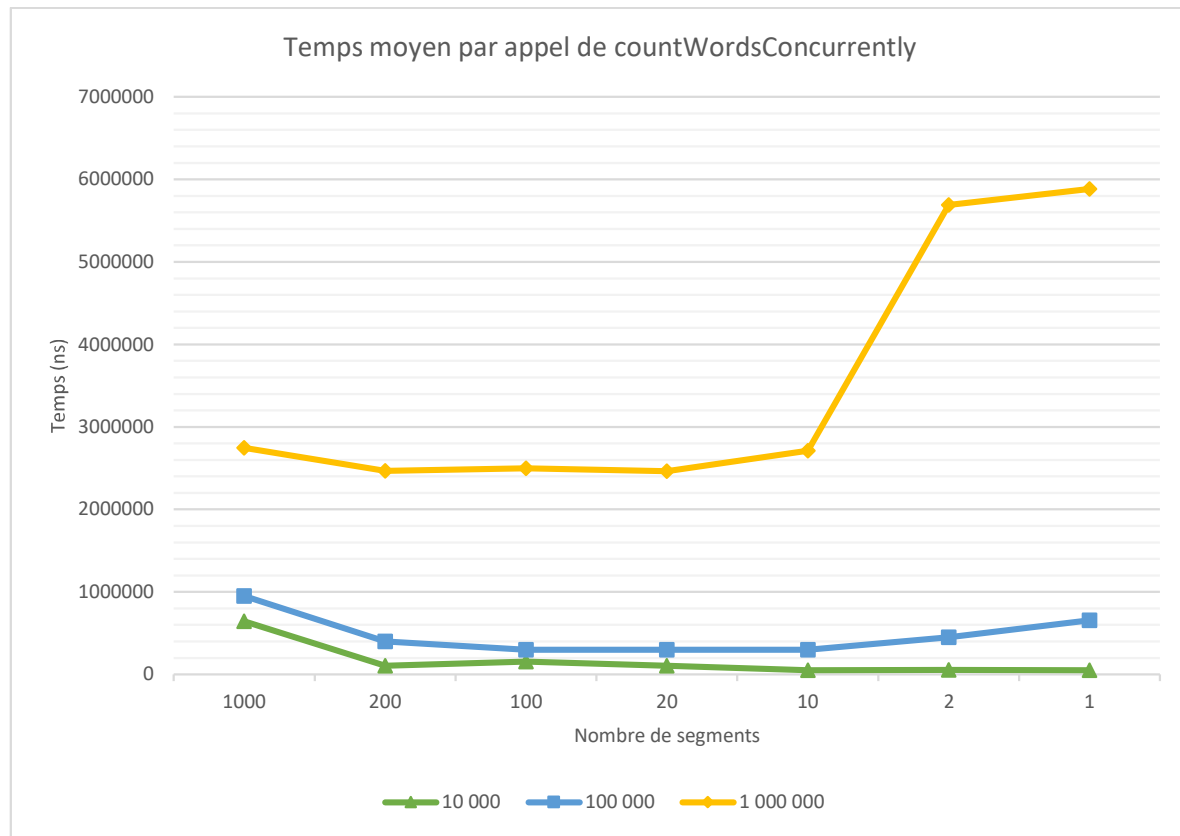
Tout d'abord, on peut observer les résultats des benchmarks. On remarque que le temps par opération est le plus petit quand le nombre de segments est très grand, mais qu'il n'est pas trop proche de la limite d'un segment par caractère. Autrement dit, des segments de plus petite taille semblent plus efficaces que les segments de grande taille, mais les plus petits segments possibles perdent en efficacité.

Taille du texte (octets)	Taille de segments	Nombre de segments	Nombre d'itérations	Temps par opération (ns/op)	Octets par opération (B/op)	Allocations par opération (allocs/op)
10000	10	1000	1809	595287	171970	4048
10000	100	100	10000	129131	80490	406
10000	1000	10	14354	89886	72634	44
10000	10000	1	9034	129335	70504	8
10000000	100	100000	6	184479017	95853602	500632
10000000	1000	10000	31	34767000	71967450	44647
10000000	10000	1000	49	24267367	69397937	4048
10000000	100000	100	45	25295622	69501361	408
10000000	1000000	10	39	30180421	68920314	44
10000000	10000000	1	16	67403931	68829470	8



Si on observe les résultats générés par la fonction `analyzePerformance()`, on peut observer l'évolution des temps passés dans `countWordsConcurrently()` pour un fichier de taille fixe si la taille des segments (et donc le nombre de segments) varie. Cette variation est appliquée sur trois tailles de fichiers différentes de telle sorte que leurs nombres de segments suivent la même échelle. Ceci nous permet d'obtenir des résultats comparables.

Taille du fichier (octets)	Taille du segment (octets)	Nombre de segments	Temps moyen (ns)	Mots/ms
10 000	10	1000	644390	1828.09
10 000	50	200	105560	11159.53
10 000	100	100	154760	7611.79
10 000	500	20	106160	11096.46
10 000	1000	10	51680	22794.12
10 000	5000	2	53160	22159.52
10 000	10000	1	51550	22851.6
100 000	100	1000	950910	12373.41
100 000	500	200	399930	29420.15
100 000	1000	100	299840	39240.93
100 000	5000	20	300530	39150.83
100 000	10000	10	300050	39213.46
100 000	50000	2	450350	26126.35
100 000	100000	1	655240	17956.78
1 000 000	1000	1000	2745430	42852.3
1 000 000	5000	200	2468140	47666.66
1 000 000	10000	100	2498930	47079.35
1 000 000	50000	20	2462810	47769.82
1 000 000	100000	10	2712350	43374.93
1 000 000	500000	2	5689590	20677.76
1 000 000	1000000	1	5883760	19995.38



En observant ces graphiques, on remarque que les fonctions ont l'air d'être à leur plus efficace lorsqu'elles opèrent sur 100 à 20 segments, peu importe la quantité de caractères que ces segments contiennent. Cependant, pour le fichier de 1 000 000, les segments plus grands ont été beaucoup plus lents que les segments plus courts. Le benchmark sur le fichier de 10Mo ne semble pas souffrir de ce ralentissement, malgré le fait qu'il contient dix fois plus de données. À l'extérieur de cette variation, toutes les données de performances semblent suivre la même tendance : la performance est optimale entre 100 et 10 segments, soit entre [100, 10] goroutines, peu importe la taille totale du texte ou celle des segments individuels.

Annexe

Annexe 1 : Résultats de benchmark 14-04-2026

Commande: go test -run='^\$' -bench='.' -benchmem -count=1 -v

```
goos: windows
goarch: amd64
pkg: TN5
cpu: 11th Gen Intel(R) Core(TM) i7-11700K @ 3.60GHz
BenchmarkSmallSegments
BenchmarkSmallSegments-16          1809          595287 ns/op      171970
B/op    4048 allocs/op
BenchmarkMediumSegments
BenchmarkMediumSegments-16         10000          129131 ns/op
80490 B/op    406 allocs/op
BenchmarkLargeSegments
BenchmarkLargeSegments-16          14354          89886 ns/op      72634
B/op     44 allocs/op
BenchmarkVeryLargeSegments
BenchmarkVeryLargeSegments-16       9034          129335 ns/op
70504 B/op     8 allocs/op
Benchmark10MBSmallSegments
Benchmark10MBSmallSegments-16        6          184479017 ns/op
95853602 B/op  500632 allocs/op
Benchmark10MBMediumSegments
Benchmark10MBMediumSegments-16       31          34767000 ns/op
71967450 B/op  44647 allocs/op
Benchmark10MBLargeSegments
Benchmark10MBLargeSegments-16        49          24267367 ns/op
69397937 B/op  4048 allocs/op
Benchmark10MBVeryLargeSegments
Benchmark10MBVeryLargeSegments-16    45          25295622 ns/op
69501361 B/op   408 allocs/op
Benchmark10MBMegaSegments
Benchmark10MBMegaSegments-16        39          30180421 ns/op
68920314 B/op   44 allocs/op
Benchmark10MBGiganticSegments
Benchmark10MBGiganticSegments-16     16          67403931 ns/op
68829470 B/op    8 allocs/op
PASS
ok      TN5      8942.967s
```

Annexe 2 : Analyse supplémentaire de la performance 10-04-2026

Remplacer le contenu de la fonction main() dans wordcount.go avec l'appel de analyzePerformance()

Commande : go run .

=== PERFORMANCE ANALYSIS ===

File Size | Segment Size | Num Segments | Avg Time (ns) | Words/ms

```
-----
10000 | 10 | 1000 | 644390.000 | 1828.09
10000 | 50 | 200 | 105560.000 | 11159.53
```

10000	100	100	154760.000	7611.79
10000	500	20	106160.000	11096.46
10000	1000	10	51680.000	22794.12
10000	5000	2	53160.000	22159.52
10000	10000	1	51550.000	22851.60
100000	100	1000	950910.000	12373.41
100000	500	200	399930.000	29420.15
100000	1000	100	299840.000	39240.93
100000	5000	20	300530.000	39150.83
100000	10000	10	300050.000	39213.46
100000	50000	2	450350.000	26126.35
100000	100000	1	655240.000	17956.78
1000000	1000	1000	2745430.000	42852.30
1000000	5000	200	2468140.000	47666.66
1000000	10000	100	2498930.000	47079.35
1000000	50000	20	2462810.000	47769.82
1000000	100000	10	2712350.000	43374.93
1000000	500000	2	5689590.000	20677.76
1000000	1000000	1	5883760.000	19995.38